

## DOCUMENTATION TECHNIQUE

# Mise en place d'un conteneur Terraform

Préparation d'un LXC Proxmox dédié à l'exécution des scripts Terraform du projet CYNA

| Auteur             | Ilyas KHALDI                          |
|--------------------|---------------------------------------|
| Date               | 19 juin 2026                          |
| Hôte Proxmox       | Nœud du labo CYNA                     |
| Conteneur LXC      | CTID 200 — Terraform-Cyna             |
| OS du conteneur    | Debian 12 (template standard Proxmox) |
| Version Terraform  | Dernière stable (repo APT HashiCorp)  |
| Provider Terraform | bpg/proxmox >= 0.66.0                 |

# 1. Objectif et contexte

Le projet CYNA repose sur une infrastructure Proxmox VE comprenant plusieurs machines virtuelles (OPNsense, Active Directory Samba, Wazuh SIEM, Bastion SSH, TrueNAS, endpoint Windows 11) et conteneurs LXC (Authelia, Nginx reverse proxy, Ansible control node, WireGuard VPN). Pour automatiser le provisionnement de cette infrastructure, l'équipe a fait le choix de l'Infrastructure as Code via Terraform et le provider bpg/proxmox.

L'objectif de ce livrable est de mettre à disposition un environnement Terraform isolé, sous la forme d'un conteneur LXC dédié hébergé directement sur le nœud Proxmox CYNA. Les scripts .tf y sont déposés et le projet est initialisé (provider téléchargé, syntaxe validée).

## 1.1 Choix techniques

Plusieurs choix structurants ont été faits :

- Conteneur LXC plutôt que VM : démarrage instantané, faible consommation mémoire (~80 Mo idle), suffisant pour un binaire Go statique comme Terraform.
- Debian 12 (Bookworm) plutôt que Debian 13 (Trixie) : le dépôt APT HashiCorp officiel supporte nativement Bookworm, alors que Trixie nécessite un contournement (forcer le codename bookworm dans la sources.list).
- Debian 12 plutôt qu'Alpine Linux : le binaire Terraform officiel est compilé contre glibc et non musl. Alpine impose soit le paquet community souvent en retard, soit l'installation de gcompat. Le gain en RAM (~50 Mo) ne justifie pas le risque de compatibilité.
- Provider bpg/proxmox plutôt que Telmate/proxmox : maintenance active, support natif des tokens API, syntaxe HCL plus moderne.
- Authentification par token API plutôt que par mot de passe : conforme aux bonnes pratiques de sécurité, révocable indépendamment du compte utilisateur.

## 2. Création du conteneur LXC

### 2.1 Téléchargement du template Debian 12

Depuis l'interface web Proxmox, sélectionner le nœud cible, puis le storage local. Onglet CT Templates → bouton Templates. Sélectionner debian-12-standard dans la liste et cliquer sur Download.

*! Si l'onglet CT Templates n'apparaît pas sur le storage, vérifier dans Datacenter → Storage → Edit que le type de contenu Container template est bien coché.*

Le template est ensuite disponible dans /var/lib/vz/template/cache/ sur le nœud.

### 2.2 Création du conteneur via la GUI

Bouton Create CT en haut à droite de l'interface Proxmox. Paramètres retenus :

| Paramètre             | Valeur                 |
|-----------------------|------------------------|
| CT ID                 | 200                    |
| Hostname              | Terraform-Cyna         |
| Template              | debian-12-standard     |
| Type                  | Unprivileged container |
| CPU cores             | 2                      |
| Mémoire               | 1024 Mo                |
| Swap                  | 512 Mo                 |
| Disque                | 8 Go (storage local)   |
| Bridge réseau         | vmbr0                  |
| IPv4                  | DHCP                   |
| Démarrage automatique | Activé                 |

## 3. Installation de Terraform

### 3.1 Prérequis

Le template Debian 12 standard de Proxmox est volontairement minimal : gnupg, wget, curl et lsb-release ne sont pas présents. Sans ces paquets, l'ajout du dépôt HashiCorp échoue et génère un fichier sources.list malformé (codename vide), bloquant ensuite tout apt update.

```
apt update
apt install -y gnupg lsb-release curl wget ca-certificates nano
```

### 3.2 Ajout du dépôt HashiCorp

Téléchargement et installation de la clé GPG officielle :

```
wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

Ajout du dépôt en référence explicite à bookworm (le codename est codé en dur plutôt que dynamique pour éviter toute cascade d'erreurs en cas d'absence de lsb-release) :

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com bookworm main" \
> /etc/apt/sources.list.d/hashicorp.list
```

### 3.3 Installation et vérification

```
apt update
apt install -y terraform
terraform -version
```

La commande `terraform -version` retourne la version installée, confirmant que le binaire est dans le PATH et exécutable.

## 4. Structure du projet Terraform CYNA

Le projet est déposé dans `/root/terraform/cyna/` sur le conteneur. Il suit la convention HashiCorp avec séparation des fichiers selon leur rôle :

```
/root/terraform/cyna/
├── main.tf           # ressources VMs et LXC
├── variables.tf      # déclaration des variables
├── outputs.tf        # récapitulatif post-déploiement
├── secrets.tfvars    # valeurs sensibles (hors Git)
└── .gitignore
```

### 4.1 main.tf — Provisionnement des ressources

Le fichier déclare le provider `bpg/proxmox` puis 10 ressources : 6 machines virtuelles (OPNsense pare-feu, Samba AD, Wazuh SIEM, Bastion SSH, TrueNAS, endpoint Win11) et 4 conteneurs LXC (Authelia, Nginx, Ansible, WireGuard). Chaque ressource précise CPU, mémoire, disque, interface réseau (avec VLAN tagging), et le cas échéant configuration d'initialisation cloud-init.

### 4.2 variables.tf — Variables d'entrée

Quatre variables sont déclarées :

| Variable                       | Type                  | Défaut    | Description                                    |
|--------------------------------|-----------------------|-----------|------------------------------------------------|
| <code>proxmox_endpoint</code>  | string                | —         | URL de l'API Proxmox                           |
| <code>proxmox_api_token</code> | string<br>(sensitive) | —         | Token API au format<br>USER@pam!TOKENID=SECRET |
| <code>proxmox_node</code>      | string                | laurent   | Nom du nœud Proxmox cible                      |
| <code>storage_pool</code>      | string                | local-zfs | Datastore principal                            |

### 4.3 outputs.tf — Récapitulatif

Définit une map `infrastructure_summary` listant chaque ressource avec son ID, son IP attribuée et son VLAN de rattachement. Affichée après l'éventuel `terraform apply`.

### 4.4 secrets.tfvars — Valeurs sensibles

Fichier exclu du dépôt Git via `.gitignore`. Contient les valeurs réelles des variables `proxmox_endpoint` et `proxmox_api_token`. Gabarit utilisé :

```
proxmox_endpoint = "https://<IP-Proxmox>:8006/"
proxmox_api_token = "terraform@pve!cyna=<UUID-secret>"
proxmox_node     = "laurent"
storage_pool     = "local-lvm"
```

*! Le token API doit être généré côté Proxmox sous Datacenter → Permissions → API Tokens, avec les droits appropriés (PVEVMAdmin sur le path /). La case Privilege Separation peut rester cochée si un rôle dédié est appliqué au token.*

## 4.5 .gitignore

```
secrets.tfvars
*.tfstate
*.tfstate.*
.terraform/
.terraform.lock.hcl
```

## 5. Initialisation du projet

Une fois les quatre fichiers .tf en place, l'initialisation est lancée depuis le dossier projet :

```
cd /root/terraform/cyna
terraform init
```

Cette commande télécharge le provider bpg/proxmox depuis le Terraform Registry et crée le dossier .terraform/ ainsi que le fichier .terraform.lock.hcl figeant la version exacte du provider.

### 5.1 Validation syntaxique

```
terraform validate
```

Vérifie la cohérence HCL du projet : syntaxe correcte, blocs bien formés, références internes valides, types respectés. Un retour Success! The configuration is valid. confirme que le projet est prêt à l'emploi.

### 5.2 État du projet après initialisation

À ce stade, le conteneur Terraform-Cyna contient :

- Un binaire Terraform installé et fonctionnel
- Le projet CYNA déposé dans /root/terraform/cyna/
- Le provider bpg/proxmox téléchargé et figé en version
- La syntaxe des scripts validée
- Un fichier .gitignore protégeant les éléments sensibles et l'état Terraform